# ECG Heartbeats

## Reading Heartbeats With AI — and Why My 'Great' Score Was Fake

*Capstone Project Report*

An AI that sorts heartbeats into four types — plus the big lesson about how a model can look amazing and actually be bad.

**RL · RLcapstone.ai**

## 1. Summary

This project takes single heartbeats from an ECG (the squiggly heart signal doctors record) and sorts each one into four types, including normal and a couple of dangerous kinds. I used a small AI model for it. But honestly, the real point of this project turned out to be a lesson about how testing can trick you.

While it was training, the model looked great. But when I tested it the honest way — on patients it had **never seen** — it got **70% right overall**, and its fair score across all four types was only **44%**. It's good at spotting one dangerous type (86%) but almost totally misses another (9%). Figuring out why is the interesting part.

## 2. Why I built this

Spotting heart-rhythm problems from an ECG is a classic AI problem, so I wanted to build the whole thing. But mostly I wanted to test it the *right* way, because a lot of projects online accidentally cheat at this and get fake-high scores.

## 3. The data

I used the MIT-BIH database, a famous free collection of heart recordings where a cardiologist labeled every single beat. I cut the signal into individual beats and sorted them into four types.

> The important choice: I made sure the patients in my test set were **completely different people** from the ones I trained on. That way the score shows how the model does on new people, not on people it already memorized.

## 4. How it works

It's a small neural network (77,000 numbers). Since about 90% of beats are normal, a lazy model would just call everything normal — so I told the model to care more about the rare, important beats, and I graded it on how well it handled all four types fairly, not just overall.

## 5. Results

Here's how it did on the patients it had never seen:

| Beat type | Caught | Note |
|---|---|---|
| Normal | 72% | The most common type |
| Supraventricular | 9% | Almost totally missed |
| Ventricular | 86% | Caught these well |
| Fusion | 9% | Very rare, very hard |

So: 70% right overall, but only **44%** once you weigh all four types fairly.

### Why it fails on some beats

A 'ventricular' bad beat looks weirdly shaped, so a model that reads shapes catches it easily. But a 'supraventricular' bad beat looks almost like a normal one — the thing that makes it abnormal is its **timing** (it comes early). My model looks at each beat by itself with no sense of timing, so it's basically blind to the exact clue it needs. That's not a bug — it's the honest limit of how I set it up.

## 6. Getting it running

I exported the model so it runs right inside a web browser (a tiny 325 KB file) and also made an Android app for it. Both give the same answers as the original. On the website you can pick a real heartbeat and watch it get sorted on your own device.

## 7. Honest limitations

- Not a medical device — just a school project on a research dataset.
- 44% isn't good enough to actually use — and showing that honestly is the whole point.
- Real ECGs are messier than the clean research ones I used.

## 8. What's next

- Add timing info (how far apart the beats are) — that's the missing clue for the type it fails on.
- Show a full breakdown of exactly what it mixes up with what.